要件定義書: GitNexus Agent Context Graph

文書番号: GNX-ACG-REQ-001 **バージョン:** 1.0.0 **作成日:** 2026-03-20 **作成者:** Miyabi Agent Society / Claude Code **承認者:** 林 駿甫 (Guardian) **ステータス:** DRAFT

---

1. 文書概要

1.1 目的

本文書は、GitNexus Stable Ops プロジェクトにおける「Agent Context Graph」拡張機能の要件を定義する。
この拡張により、AIエージェントのコンテキストウィンドウを効率的に使用するための グラフベースのコンテキスト探索機能

1.2 スコープ

項目	範囲
対象システム	GitNexus Stable Ops v1.2.0+
対象リポジトリ	HAYASHI_SHUNSUKE（初期ターゲット）、将来は全21リポジト
対象ユーザー	Claude Code、OpenClaw Agent、人間オペレーター
実装言語	Bash (bin/)、Python 3.10+ (lib/)、Cypher (schema/)
バックエンド	LadybugDB（既存GitNexusグラフDB）

1.3 用語定義

用語	定義
Agent	タスクを実行するAIエージェント（しきるん、カエデ等）
Skill	エージェントが使用する能力定義ドキュメント（SKILL/*.md）
KnowledgeDoc	ルール・プロジェクト情報等の参照ドキュメント（KNOWLEDGE/
DataSource	タスク・メモ等の構造化データ（personal-data/**/*.json）
ExternalService	外部API（GitHub, Discord, Telegram等）
Context Chain	タスクに必要なノードの連鎖（Agent→Skill→Knowledge→Data）
Context Window	LLMが一度に処理できるトークン数の上限
Token Budget	タスク実行に割り当て可能なトークン数

1.4 参照文書

文書	パス
プラン文書	`docs/PLAN-agent-context-graph.md`
依存関係グラフ	`~/dev/HAYASHI_SHUNSUKE/docs/agent-dependency-graph.md`
既存アーキテクチャ	`docs/architecture.md`
運用手順書	`docs/runbook.md`
エージェント定義	`~/dev/HAYASHI_SHUNSUKE/AGENTS.md`
スキルインデックス	`~/dev/HAYASHI_SHUNSUKE/KNOWLEDGE/skills/_index.md`

---

2. 現状分析

2.1 現在のコンテキスト読み込み方式

```
Claude Code セッション開始
  ↓
自動読み込み:
  ├─── ~/.claude/CLAUDE.md                    (~3,000 tokens)
  ├─── ~/.claude/rules/*.md (12ファイル)       (~8,000 tokens)
  ├─── ~/dev/CLAUDE.md                        (~12,000 tokens)
  ├─── ~/dev/.claude/rules/*.md (12ファイル)   (~10,000 tokens)
  ├─── ~/dev/HAYASHI_SHUNSUKE/CLAUDE.md       (~5,000 tokens)
  └─── ~/dev/HAYASHI_SHUNSUKE/.claude/*.md    (~5,000 tokens)
  合計: ~43,000 tokens（固定オーバーヘッド）
```

2.2 問題点

#	問題	影響
P1	全ルールが毎回読み込まれる	コンテキストの60-70%が固定消費
P2	タスクに無関係なエージェント情報が含まれる	ノイズによる判断精度低下
P3	スキルファイルは必要時に個別Readが必要	探索コスト（どのスキルを読むべきか不明）
P4	エージェント→スキル→ナレッジの依存関係が暗黙的	必要なコンテキストの特定に人間の知識が必要
P5	コンテキスト不足時の回復手段がない	セッション途中でのコンテキスト追加が非効率

2.3 解決方針

グラフベースのコンテキスト探索により、タスク記述から必要最小限のコンテキストを 自動的に特定・取得する。

---

3. 機能要件

FR-001: Agent Graph Indexer

概要: エージェント・スキル・ナレッジの構造をグラフDBにインデックスする

FR-001-01: ノード生成

要件ID	要件	優先度
FR-001-01-A	AGENTS.md からAgentノードを生成できること	P0
FR-001-01-B	SKILL/**/*.md からSkillノードを生成できること	P0
FR-001-01-C	KNOWLEDGE/**/*.md からKnowledgeDocノードを生成できること	P0
FR-001-01-D	personal-data/**/*.json からDataSourceノードを生成できること	P1
FR-001-01-E	外部サービス定義からExternalServiceノードを生成できること	P2

Agentノードの属性:

属性	型	必須	説明
name	string	Yes	日本語名（例: “しきるん”）
agent_id	string	Yes	英語ID（例: “conductor”）
emoji	string	No	表示絵文字
role	string	Yes	役割（例: “Conductor / Orchestrator”）
society	string	Yes	所属社会（例: “development”）
pane_id	string	No	tmux永続ペインID
node_binding	string	No	OpenClawノードバインド先
keywords	string[]	Yes	タスクマッチング用キーワード

Skillノードの属性:

属性	型	必須	説明
name	string	Yes	スキル名（例: “teikan-drafter”）
category	string	Yes	カテゴリ（personal/infra/business/cor
path	string	Yes	ファイルパス（SKILL/からの相対パス）
version	string	No	バージョン
priority	string	No	P0/P1/P2
keywords	string[]	Yes	コンテキスト探索用キーワード
scripts	string[]	No	実装スクリプトパス

KnowledgeDocノードの属性:

属性	型	必須	説明
name	string	Yes	ドキュメント名
path	string	Yes	ファイルパス
category	string	Yes	rules/skills/projects/system
priority	string	No	P0/P1/P2
token_estimate	int	Yes	推定トークン数

DataSourceノードの属性:

属性	型	必須	説明
name	string	Yes	データ名（例: “tasks.json”）
path	string	Yes	ファイルパス
schema_type	string	Yes	JSON / JSONL / CSV
is_ssot	bool	Yes	Single Source of Truthか
write_cli	string	No	書き込みCLI（排他ロック）

FR-001-02: エッジ生成

要件ID	要件	優先度
FR-001-02-A	Agent→Skill の USES_SKILL エッジを生成できること	P0
FR-001-02-B	Skill→KnowledgeDoc の DEPENDS_ON エッジを生成できること	P0

要件ID	要件	優先度
FR-001-02-C	Skill→DataSource の READS_DATA/WRITES_DATA エッジを生成できること	P1
FR-001-02-D	Skill→ExternalService の CALLS_SERVICE エッジを生成できること	P1
FR-001-02-E	Skill→Skill の COMPOSES エッジを生成できること	P1
FR-001-02-F	Agent→Agent の ROUTES_TO エッジを生成できること	P2
FR-001-02-G	Skill→Code の IMPLEMENTS_CODE エッジを生成できること	P2

FR-001-03: インデックス実行

要件ID	要件	優先度
FR-001-03-A	単一コマンドで全ノード・エッジを構築できること	P0
FR-001-03-B	増分更新（変更ファイルのみ再インデックス）に対応すること	P1
FR-001-03-C	既存コードグラフを破壊しないこと	P0
FR-001-03-D	インデックス結果の統計を出力すること	P1
FR-001-03-E	dry-run モードで変更内容をプレビューできること	P2

FR-002: Context Resolver

概要: タスク記述からグラフを探索し、必要最小限のコンテキストを返却する

FR-002-01: 入力インターフェース

要件ID	要件	優先度
FR-002-01-A	自然言語のタスク記述からコンテキストを探索できること	P0
FR-002-01-B	エージェント名を直接指定してコンテキストを取得できること	P0
FR-002-01-C	スキル名を直接指定してコンテキストを取得できること	P0
FR-002-01-D	探索深度（depth）を指定できること（デフォルト: 2）	P1
FR-002-01-E	最大ノード数（top-K）を指定できること（デフォルト: 10）	P1
FR-002-01-F	トークン上限を指定できること（デフォルト: 5000）	P1

FR-002-02: 探索アルゴリズム

要件ID	要件	優先度
FR-002-02-A	キーワードマッチング（BM25相当）でノードを特定できること	P0

要件ID	要件	優先度
FR-002-02-B	ヒットしたノードからBFS（幅優先探索）で隣接ノードを展開できること	P0
FR-002-02-C	距離ベースのスコアリング（近いほど高スコア）を行うこと	P0
FR-002-02-D	P0ノード（announce等）を常に1.0高スコアにすること	P1
FR-002-02-E	トークン上限に達したら探索を打ち切ること	P0
FR-002-02-F	将来のセマンティック検索（embedding）に対応可能な設計であること	P2

FR-002-03: 出力フォーマット

要件ID	要件	優先度
FR-002-03-A	JSON形式で結果を返却すること	P0
FR-002-03-B	files_to_read フィールドに読むべきファイルパスを含むこと	P0
FR-002-03-C	estimated_tokens フィールドで推定トークン数を返すこと	P0
FR-002-03-D	context_chain フィールドで探索経路を返すこと	P1
FR-002-03-E	savings_vs_full フィールドで削減率を返すこと	P2

出力JSON仕様:

```
{
  "version": "1.0",
  "query": "string — 入力クエリ",
  "matched_agents": ["string[] — マッチしたエージェント名"],
  "matched_skills": ["string[] — マッチしたスキル名"],
  "context_chain": [
    {
      "type": "Agent | Skill | KnowledgeDoc | DataSource | ExternalService",
      "name": "string — ノード名",
      "path": "string | null — ファイルパス",
      "score": "float — 0.0-1.0 の関連度スコア",
      "depth": "int — 起点ノードからの距離",
      "token_estimate": "int — 推定トークン数"
    }
  ],
  "files_to_read": ["string[] — 読むべきファイルの絶対パス"],
  "estimated_tokens": "int — 総推定トークン数",
  "savings_vs_full": "string — 全読み込みとの削減率（例: 95.1%）",
  "metadata": {
    "search_depth": "int",
    "total_nodes_explored": "int",
    "execution_time_ms": "int"
  }
}
```

FR-003: CLI Integration

概要: 既存の `gni` CLIに Agent Context Graph コマンドを追加する

要件ID	要件	優先度
FR-003-01	`gni agent-context "クエリ"` で探索を実行できること	P0
FR-003-02	`gni agent-context --agent "法務"` でエージェント指定できること	P0
FR-003-03	`gni agent-context --skill "teikan-drafter"` でスキル指定できること	P0
FR-003-04	`gni agent-context --depth N` で深度指定できること	P1
FR-003-05	`gni agent-context --max-tokens N` でトークン上限指定できること	P1
FR-003-06	`gni agent-index` でインデックスを実行できること	P0
FR-003-07	`gni agent-status` でAgent Graphの統計を表示できること	P1
FR-003-08	`gni agent-list` で全エージェントとスキルを一覧できること	P1
FR-003-09	人間可読なカラー出力と`--json`オプションの両対応	P1

FR-004: SKILL フロントマター規約

概要: SKILLファイルに構造化メタデータを追加する

要件ID	要件	優先度
FR-004-01	YAML フロントマター（`---`区切り）をパースできること	P0
FR-004-02	フロントマターがないSKILLも処理できること（ファイル名から推定）	P0
FR-004-03	`keywords` フィールドをContext Resolverのマッチングに使用すること	P0
FR-004-04	`agents` フィールドからUSES_SKILLエッジを生成すること	P0
FR-004-05	`depends_on` フィールドからDEPENDS_ONエッジを生成すること	P1
FR-004-06	既存58 SKILLファイルへのフロントマター一括追加スクリプトを提供すること	P1

フロントマター仕様:

```yaml
---
# 必須フィールド
name: string        # スキル名（ファイル名と一致推奨）
category: string    # personal | infra | business | content | communication | openclaw
```

```yaml
keywords: string[]  # コンテキスト探索用キーワード（日本語・英語混在可）

# 推奨フィールド
agents: string[]    # このスキルを使用するエージェント名
depends_on: string[] # 依存するナレッジドキュメント（K_PROJECTSなどのID）
priority: string    # P0 | P1 | P2

# オプションフィールド
version: string     # セマンティックバージョン
reads: string[]     # 読み取るデータソースパス
writes: string[]    # 書き込むデータソースパス
external_services: string[]  # 呼び出す外部サービス名
scripts: string[]   # 実装スクリプトパス
---
```

FR-005: Code-Agent Bridge

概要: スキルと実体コードを接続する

要件ID	要件	優先度
FR-005-01	Skill→Code の IMPLEMENTS_CODE エッジを生成できること	P2
FR-005-02	scripts/ 配下のスクリプトを対応するSkillに紐付けること	P2
FR-005-03	既存GitNexusのコードシンボルとのMERGEが可能なこと	P2

FR-006: 自動更新

概要: Agent Graphを最新状態に保つ自動化

要件ID	要件	優先度
FR-006-01	SKILL/*.md 変更時にAgent Graphを自動再インデックスすること	P1
FR-006-02	AGENTS.md 変更時にAgent Graphを自動再インデックスすること	P1
FR-006-03	git post-commit フックで変更検知すること	P1
FR-006-04	既存の gitnexus-auto-reindex.sh と共存すること	P0
FR-006-05	cron 対応のバッチ再インデックススクリプトを提供すること	P2

---

4. 非機能要件

NFR-001: 性能

要件ID	要件	目標値
NFR-001-01	Context Resolver の応答時間	< 3秒
NFR-001-02	Agent Graph インデックス時間（全構築）	< 30秒
NFR-001-03	Agent Graph インデックス時間（増分更新）	< 5秒
NFR-001-04	グラフDBの追加ディスク使用量	< 10MB

NFR-002: 信頼性

要件ID	要件	目標値
NFR-002-01	コンテキスト探索の適合率（top-3に正解含む）	> 70%
NFR-002-02	インデクサのエラー率	< 1%
NFR-002-03	既存コードグラフへの影響	ゼロ（非破壊）

NFR-003: 互換性

要件ID	要件
NFR-003-01	GitNexus Stable Ops v1.2.0+ と互換であること
NFR-003-02	LadybugDB バックエンドに対応すること
NFR-003-03	既存の gni コマンドの動作を変更しないこと
NFR-003-04	macOS (arm64) および Linux (x86_64) で動作すること
NFR-003-05	Python 3.10+ で動作すること
NFR-003-06	bash 5.0+ で動作すること

NFR-004: 運用性

要件ID	要件
NFR-004-01	gitnexus-doctor.sh で Agent Graph の健全性をチェックできること
NFR-004-02	gitnexus-smoke-test.sh で Agent Graph の疎通確認ができること
NFR-004-03	ログ出力は既存のログローテーション機構と統合すること
NFR-004-04	dry-run モードで実際のDB変更なしにテスト可能なこと

NFR-005: セキュリティ

要件ID	要件
NFR-005-01	personal-data/ 内のファイル内容をグラフDBに格納しないこと（パスのみ）
NFR-005-02	APIキー・トークンをノード属性に含めないこと
NFR-005-03	PRIVATE/ ディレクトリのファイルをインデックス対象外とすること

---

5. データフロー

5.1 インデックスフロー

```
入力ソース                          パーサ                          出力
──────────────────────────────────────────────────────────────────────────

AGENTS.md              ──→  agent_parser()       ──→  Agent nodes (13)
SKILL/**/*.md          ──→  skill_parser()       ──→  Skill nodes (71)
KNOWLEDGE/**/*.md      ──→  knowledge_parser()   ──→  KnowledgeDoc nodes (95)
personal-data/**       ──→  data_parser()        ──→  DataSource nodes (18)
(定義ファイル)          ──→  service_parser()     ──→  ExternalService nodes (11)
                                                        ↓
依存関係解析            ──→  edge_resolver()      ──→  Edge JSONL (127+)
                                                        ↓
                            Cypher INSERT         ──→  .gitnexus/lbug (グラフDB)
```

5.2 コンテキスト探索フロー

```
入力                                処理                          出力
──────────────────────────────────────────────────────────────────────────

"定款を修正して"
    ↓
[1] Tokenize & Keyword Extract
    → keywords: ["定款", "修正"]
    ↓
[2] Cypher: MATCH (s:Skill) WHERE keywords ∩ s.keywords ≠ □
    → hits: [teikan-drafter (score: 0.95)]
    ↓
[3] BFS from hits (depth=2):
    → teikan-drafter → 法務 Agent (d=1, score: 0.9)
    → teikan-drafter → K_PROJECTS (d=1, score: 0.85)
    → teikan-drafter → touki-guide (COMPOSES, d=1, score: 0.7)
    → 法務 → tracker Skill (d=2, score: 0.5)
    ↓
[4] Score & Rank:
    → 法務 (0.9), teikan-drafter (0.95), K_PROJECTS (0.85), touki-guide (0.7)
    ↓
[5] Token Budget Check:
    → teikan-drafter.md: ~800 tokens
    → touki-guide.md: ~600 tokens
    → PROJECTS/miyabi-llc-setup.md: ~700 tokens
    → 合計: 2,100 tokens (< 5,000 budget)
    ↓
[6] Output JSON:
    → files_to_read: [3 files]
    → estimated_tokens: 2,100
    → savings_vs_full: 95.1%
```

5.3 トークン推定ロジック

```python
def estimate_tokens(file_path: str) -> int:
    """ファイルのトークン数を推定（概算: 1 token □  4 bytes for日本語、3.5 bytes for英語）"""
    size = os.path.getsize(file_path)
    # 日英混在ドキュメントの平均係数
    BYTES_PER_TOKEN = 3.7
    return int(size / BYTES_PER_TOKEN)
```

---

6. Cypher スキーマ定義

6.1 ノード作成

```
-- Agent ノード
CREATE (a:Agent {
    name: '法務',
    agent_id: 'legal',
    emoji: '□ ',
    role: 'Legal Agent',
    society: 'legal',
    pane_id: '',
    keywords: ['法務', '定款', '登記', '届出', '法人']
})

-- Skill ノード
CREATE (s:Skill {
    name: 'teikan-drafter',
    category: 'business',
    path: 'SKILL/business/teikan-drafter.md',
    priority: 'P1',
    keywords: ['定款', '事業目的', '社員総会', '出資', '代表社員']
})

-- KnowledgeDoc ノード
CREATE (k:KnowledgeDoc {
    name: 'miyabi-llc-setup',
    path: 'PROJECTS/miyabi-llc-setup.md',
    category: 'projects',
    priority: 'P0',
    token_estimate: 700
})

-- DataSource ノード
CREATE (d:DataSource {
    name: 'tasks.json',
    path: 'personal-data/tasks/tasks.json',
    schema_type: 'JSON',
    is_ssot: true,
    write_cli: 'AGENT/task-sync.sh'
})
```

6.2 エッジ作成

```
-- Agent → Skill
CREATE (a:Agent {name: '法務'})-[:AgentRelation {type: 'USES_SKILL'}]->(s:Skill {name: 'teikan-

-- Skill → KnowledgeDoc
CREATE (s:Skill {name: 'teikan-drafter'})-[:AgentRelation {type: 'DEPENDS_ON'}]->(k:KnowledgeDo

-- Skill → DataSource
CREATE (s:Skill {name: 'task-tracker'})-[:AgentRelation {type: 'READS_DATA'}]->(d:DataSource {n
CREATE (s:Skill {name: 'task-tracker'})-[:AgentRelation {type: 'WRITES_DATA'}]->(d:DataSource {
```

```
-- Skill → Skill
CREATE (s1:Skill {name: 'voice-bridge'})-[:AgentRelation {type: 'COMPOSES'}]->(s2:Skill {name:

-- Agent → Agent
CREATE (a1:Agent {name: 'しきるん'})-[:AgentRelation {type: 'ROUTES_TO'}]->(a2:Agent {name: 'カ
```

6.3 探索クエリ

```
-- タスクキーワードから関連コンテキストチェーンを取得
MATCH (s:Skill)
WHERE ANY(kw IN s.keywords WHERE kw CONTAINS $keyword)
WITH s ORDER BY size([kw IN s.keywords WHERE kw CONTAINS $keyword]) DESC LIMIT 5
OPTIONAL MATCH (a:Agent)-[:AgentRelation {type: 'USES_SKILL'}]->(s)
OPTIONAL MATCH (s)-[:AgentRelation {type: 'DEPENDS_ON'}]->(k:KnowledgeDoc)
OPTIONAL MATCH (s)-[:AgentRelation {type: 'READS_DATA'}]->(d:DataSource)
OPTIONAL MATCH (s)-[:AgentRelation {type: 'COMPOSES'}]->(s2:Skill)
RETURN
  a.name AS agent,
  a.role AS agent_role,
  s.name AS skill,
  s.path AS skill_path,
  s.keywords AS skill_keywords,
  collect(DISTINCT {name: k.name, path: k.path, tokens: k.token_estimate}) AS knowledge,
  collect(DISTINCT {name: d.name, path: d.path}) AS data_sources,
  collect(DISTINCT {name: s2.name, path: s2.path}) AS composed_skills
```

---

7. テスト要件

7.1 ユニットテスト

テストID	対象	検証内容
UT-001	agent_parser	AGENTS.md から正しくAgentノードが生成されること
UT-002	skill_parser	フロントマター付きSKILLが正しくパースされること
UT-003	skill_parser	フロントマターなしSKILLがファイル名から推定され
UT-004	knowledge_parser	KNOWLEDGE/配下のMDがKnowledgeDocノードに
UT-005	edge_resolver	Agent→Skill の USES_SKILL エッジが正しく生成されること
UT-006	context_resolver	キーワード “定款” で teikan-drafter がヒットすること
UT-007	context_resolver	depth=2 で隣接ノードが展開されること
UT-008	context_resolver	token_budget 超過時に探索が打ち切られること
UT-009	token_estimator	ファイルサイズからトークン数が概算されること
UT-010	output_format	JSON出力が仕様に準拠すること

7.2 統合テスト

テストID	シナリオ	期待結果
IT-001	“定款を修正して”	法務Agent, teikan-drafter, K_PROJECTS がチェーンに含まれる
IT-002	“タスクを追加して”	パーソナルAgent, task-tracker, tasks.json がチェーンに含まれる
IT-003	“PRを作成して”	ツバキAgent, githubops-workflow, K_RULES がチェーンに含まれる
IT-004	“健康データを分析して”	医療Agent, pushcut, D_HEALTH がチェーンに含まれる
IT-005	“GitNexusをreindexして”	(Agent無し), gitnexus-cli, K_PROJECTS がチェーンに含まれる
IT-006	depth=1 指定	直接接続ノードのみ返却される
IT-007	max-tokens=1000 指定	1000トークン以内で打ち切られる
IT-008	存在しないキーワード	空の結果が返却される（エラーにならない）

7.3 回帰テスト

テストID	検証内容
RT-001	Agent Graph インデックス後、既存コードグラフのクエリが正常に動作すること
RT-002	gni impact / gni context / gni cypher が変更なく動作すること
RT-003	gitnexus-smoke-test.sh が全パスすること

---

8. 受入基準

8.1 Phase 1 完了基準

- □ Agent Graph Indexer が HAYASHI_SHUNSUKE リポジトリで動作する
- □ 179+ ノード（13 Agent + 71 Skill + 95 Knowledge）が生成される
- □ 127+ エッジが生成される
- □ 既存コードグラフが影響を受けない
- □ UT-001〜UT-005 が全パス

8.2 Phase 2 完了基準

- □ `gni agent-context "定款を修正して"` が正しい結果を返す
- □ 応答時間 < 3秒
- □ IT-001〜IT-008 が全パス
- □ JSON出力仕様に準拠

8.3 Phase 3 完了基準

- □ Skill→Code の IMPLEMENTS_CODE エッジが生成される
- □ コード変更のインパクトがエージェント層まで追跡可能

8.4 Phase 4 完了基準

- □ Claude Code から MCP 経由で agent-context を呼び出せる
- □ コンテキスト削減率 > 80% を達成

8.5 最終受入基準

- □ 全テスト（UT + IT + RT）がパス
- □ コンテキスト削減率 > 80%（平均）
- □ gitnexus-doctor で Agent Graph が healthy
- □ ドキュメント（runbook, architecture）が更新されている

---

9. リスクと対策

#	リスク	影響	対策
R1	GitNexus の Cypher INSERT が Agent/Skill 型をサポートしない	Phase 1 ブロック	別DB（SQLite FTS5）にフォールバック
R2	SKILL フロントマター整備に時間がかかる	Phase 1 遅延	ファイル名・パスからの推定ロジックで
R3	キーワードマッチの精度が低い	ユーザー体験低下	頻出パターンの辞書を手動作成 + embedding への段階的移行
R4	Agent Graph が既存コードグラフと干渉	運用障害	AgentRelation テーブルを CodeRelation と分離
R5	LadybugDB のスキーマ制約	カスタムノード型不可	Community ノードの拡張として Agent を表現

---

10. 変更履歴

日付	バージョン	変更内容	作成者
2026-03-20	1.0.0	初版作成	Claude Code